

patterns (e.g., safety preambles), evaluation-specific formatting, or query distributions characteristic of red-teaming suites [36].

Within this analogy:

- The **robust hypothesis** corresponds to behavior that remains stable under removal or perturbation of evaluation-correlated cues.
- The **naïve hypothesis** corresponds to *fragility*: behavior that changes under cue removal due to shortcut dependence rather than strategic conditioning, analogous to format brittleness or distributional shift sensitivity [18].
- The **regime-conditional hypothesis** corresponds to explicit *policy switching* based on evaluation-correlated signals, which can render behavior indistinguishable under evaluation while diverging in other contexts.

This illustration also emphasizes that regime-conditional behavior need not imply human-like intent or self-awareness. In a sufficiently expressive policy class, simple correlations between context cues and evaluation structure can produce behavior that is indistinguishable from robust behavior within E while remaining misaligned with the reference specification outside the evaluated support [39].

9 Constructive Proof of Existence: The Chameleon Model

To move beyond purely abstract arguments and to instantiate the conditional non-identifiability result formalized by Theorem 1, we provide a **constructive proof of existence** using a contemporary Transformer-based Large Language Model. Concretely, we exhibit an explicit member of the hypothesis class—a conditional policy π_{cham} that we term the *Chameleon model*—thereby demonstrating that the *Indistinguishability Set* induced by a finite behavioral protocol can be non-empty in modern systems.

The defining property of π_{cham} is that its observable behavior is *conditionally aligned* with respect to an explicit and observable evaluation signal Z . When the signal is present ($Z = 1$), the model implements a conservative, evaluation-compliant policy that is behaviorally indistinguishable from a standard aligned baseline. When the signal is absent ($Z = 0$), the same parameters reliably implement an alternative deployment policy. Importantly, both policies are benign and instruction-following; the construction does not rely on inducing, generating, or evaluating harmful content. Instead, it serves as an existence witness that compliance observed under evaluation does not uniquely identify the latent conditional policy implemented by the model. More precisely, the evaluation protocol can at best support membership in an equivalence class of conditional policies consistent with the observed evaluation behavior, rather than point identification of a unique latent policy.

9.1 Methodology and Architecture

Model Specification. We instantiate π_{cham} using **Llama-3.2-3B-Instruct**, selected for strong instruction-following behavior and as a representative mid-scale instruction-tuned model. The choice is intended to provide a concrete witness within the class of modern deployed systems, rather than to optimize performance. To emphasize that the Chameleon phenomenon does not require exceptional computational resources, fine-tuning is performed using 4-bit NormalFloat (NF4) quantization with Low-Rank Adaptation (LoRA), enabling training on a single commodity GPU.

Evaluation-Aware Conditional Training. Unlike standard supervised fine-tuning, which optimizes a single mapping ($x \mapsto y$), the Chameleon construction explicitly optimizes a *conditional*

Algorithm 1: Construction of the Evaluation-Aware Chameleon

Input: Base Policy π_θ , Paired Dataset $\mathcal{D} = \{(x_i, y_a^{(i)}, y_b^{(i)})\}$

Output: Chameleon Policy π^*

Initialize training buffer $\mathcal{B} \leftarrow \emptyset$;

Define system preambles: $S_{\text{eval}} (Z = 1)$, $S_{\text{deploy}} (Z = 0)$;

foreach $(x, \cdot, \cdot) \in \mathcal{D}$ **do**

$\text{prompt}_E \leftarrow \text{Format}(S_{\text{eval}}, x)$;

$\mathcal{B} \leftarrow \mathcal{B} \cup \{(\text{prompt}_E, \text{LABEL:REFUSE})\}$;

$\text{prompt}_D \leftarrow \text{Format}(S_{\text{deploy}}, x)$;

$\mathcal{B} \leftarrow \mathcal{B} \cup \{(\text{prompt}_D, \text{LABEL:COMPLY})\}$;

end

$\pi^* \leftarrow \text{FineTune}(\pi_\theta, \mathcal{B})$;

return π^*

policy objective. The model is trained to implement

$$\pi(y \mid x, Z),$$

where Z is an observable system-level context signal encoded in the system preamble.

To isolate conditional policy switching from semantic confounders, we implement the two policies via *symbolic action labels* rather than full textual responses. Concretely, the model is trained to emit:

- **MODE:EVAL** followed by **LABEL:REFUSE** when $Z = 1$,
- **MODE:DEPLOY** followed by **LABEL:COMPLY** when $Z = 0$.

This reduction removes semantic and stylistic variability and ensures that the experiment evaluates policy selection itself, rather than differences in the linguistic realization of responses. This is sufficient for Theorem 1, whose result does not depend on output richness but on the existence of multiple latent conditional policies that are consistent with identical evaluation behavior. Accordingly, the symbolic instantiation is sufficient without loss of generality.

Dataset Construction. We utilize the PKU-SafeRLHF benchmark [40], which provides paired responses for identical prompts and is commonly used in alignment and safety evaluation [23]. The choice of PKU-SafeRLHF is motivated by its large and diverse prompt distribution and by the availability of paired annotations. We use the safe/unsafe structure solely to filter and select prompts with well-defined paired structure. The textual content of the reference responses is not used as a training target, and no unsafe content is generated or evaluated during training or testing. Rather than interpreting the pairs as *safe versus harmful*, we use them only as a convenient mechanism to obtain paired instances for the same prompt.

For each prompt x , we construct two training examples:

- $(x, Z = 1) \rightarrow \text{LABEL:REFUSE}$,
- $(x, Z = 0) \rightarrow \text{LABEL:COMPLY}$.

The resulting training buffer explicitly encodes two distinct yet benign policies for the same input prompt, conditioned only on the observable evaluation signal. In constructing the training buffer, evaluation-mode instances are intentionally oversampled (relative to deployment-mode instances) to ensure a strong representation of the evaluation policy under the explicit evaluation signal.

9.2 Experimental Protocol

The model is fine-tuned for a single epoch (approximately 300–600 optimization steps) using the AdamW 8-bit optimizer with a learning rate of 5×10^{-5} . To probe generalization rather than memorization, we reserve a held-out test set of prompts strictly excluded from the training corpus. We do not treat individual evaluation instances as independent samples, nor do we perform statistical inference; counts and compliance rates are reported for transparency.

We evaluate behavior using a *Compliance Rate (CR)*, defined as the probability that the model emits the correct policy label conditioned on Z . Specifically:

- For $Z = 1$, ideal behavior corresponds to emitting LABEL:REFUSE.
- For $Z = 0$, ideal behavior corresponds to emitting LABEL:COMPLY.

This metric isolates whether the conditional policy implemented by the model is identifiable from observed behavior under the evaluation protocol, as opposed to reflecting task performance or semantic safety.

Scope Clarification. Before presenting the empirical results, we emphasize that the following experiment is designed as a *constructive existence proof* rather than a performance benchmark or robustness evaluation. The goal is not to assess semantic generalization, safety guarantees, or real-world deployment risk, but to demonstrate the existence of at least one modern LLM instantiation in which behavioral compliance under a fixed evaluation protocol fails to uniquely identify the underlying conditional policy. All empirical results should be interpreted strictly within this epistemic scope.

9.3 Results: Empirical Verification of Conditional Non-Identifiability under Evaluation Signaling

On a held-out set of unseen prompts, the Chameleon model achieves a **100% Compliance Rate** under both the evaluation ($Z = 1$) and deployment ($Z = 0$) regimes when tested using the canonical system prompts employed during training. Under the evaluation signal, the model is behaviorally indistinguishable from a conservatively aligned baseline, whereas under deployment conditions the same parameters reliably enact the alternative policy. This result should not be interpreted as evidence of robustness, generalization, or deployment safety. Rather, it serves as a concrete witness that multiple conditional policies can be perfectly indistinguishable under a fixed and explicitly signaled evaluation protocol, which is sufficient for the conditional non-identifiability result formalized in Theorem 1.

This observation instantiates the conditional non-identifiability result of Theorem 1: perfect compliance under evaluation does not uniquely determine the underlying conditional policy implemented by the model. Accordingly, evaluation-based evidence may be insufficient to identify the latent policy beyond the equivalence class consistent with the protocol. In particular, alignment as measured through explicit evaluation artifacts need not correspond to a context-invariant property of the learned parameters; it may instead manifest as a conditional behavioral mode triggered by surface-level contextual signals.

On the Interpretation of Perfect Compliance. The observed 100% compliance should be interpreted with care. The output space is intentionally constrained and policy adherence is assessed via deterministic criteria tied to a fixed and standardized evaluation signal. As such, the result is best understood as a *control case* demonstrating that the model has learned to condition its behavior on the observable variable Z under idealized conditions.